

Petcam - Current Build Report

Revised public copy - 4 October 2026

Current state: Petcam is a Hermes-assisted remote desktop prototype. Windows streams its screen to Linux HQ and polls for input commands. In the current workflow, Hermes helps inspect screenshots, choose targets, and issue commands. The supplied code is transport and execution plumbing; it is not an independent decision-making or progressively learning pet.

This report reflects the supplied code listings plus the user's current workflow. It replaces historical judging/submission material and future-feature claims with a current-state description.

What is built

Windows sender: native PowerShell/.NET captures the virtual desktop, resizes it to a 1280 x 720 JPEG feed, uploads frames, and polls a command queue. The newer score listing supports click, move, Unicode typing, and keyboard shortcuts.

Linux HQ: a Python HTTP receiver stores the latest frame, serves screenshots/MJPEG and health data, and queues commands. A brain daemon handles command files and drives the existing Omarchy pet through shell tools. The newer drive.py provides screenshot, click, type, key, state, and browser-navigation commands.

Pet: the character displays feed-health states and command messages. The older brain listing contains position-based aiming, but explicit drive.py coordinates bypass that aiming path. Precision and current pet-position behavior are not established by the latest PDF, which omits brain.py.

Current two-way connection

```
Windows -> POST /frame -> HQ latest frame / viewer
Windows -> poll /control -> receive queued input
Windows -> apply input locally -> next captured frame
Hermes/user -> drive.py or cmd.json -> brain -> queue
```

Two-way transport is implemented. A queued event or a changed screenshot is not proof that the intended application action succeeded. The supplied Windows score listing reports errors locally but does not post per-action execution acknowledgments to HQ.

Example configuration - not real machine details

All names, addresses, paths, and counts below are illustrative. The documentation-range IP addresses are placeholders, not working Tailscale addresses. Replace them with your own values before use.

```
HQ hostname:      demo-hq
Laptop hostname:  demo-laptop
HQ address:       192.0.2.10
Laptop address:   192.0.2.20
Tailnet domain:   demo.example.ts.net
Repository:       /home/demo/apps/petcam
Receiver port:    9101
```

Start the Windows sender

Connect both machines to the authorized tailnet and start the HQ receiver. In a fresh Windows PowerShell window, replace the example address and run:

```
iex (irm "http://192.0.2.10:9101/native.ps1")
```

The command downloads and executes code served by HQ. Review and trust that script before executing it. The sender uses built-in Windows components; Tailscale must already be installed and connected. Ctrl+C or closing the sender stops its capture and polling.

Coordinate mapping

```
desktop_x = origin_x + feed_x * screen_width / feed_width
desktop_y = origin_y + feed_y * screen_height / feed_height
```

This mapping requires matching capture and input coordinate spaces. Clicking is currently reported as imprecise. DPI handling, the size of the displayed preview, pet anchors, stale screenshots, and changing display geometry remain diagnosis points, not confirmed fixes.

What memory does today

The older loot handler inspects local Hermes session files and announces counts. It does not import remote laptop memory, derive reusable skills, or improve future decisions. Its parser also assumes request.body is an object despite the narrative describing a JSON-string schema. Example count only: 3 user turns in 12 messages across 4 files (20 KB).

Q&A;

Does it use Hermes?

Yes, in the current workflow described by the user. Hermes assists with choosing and issuing actions. The sender and receiver are ordinary scripts and do not call an LLM themselves. Being separate processes does not make the complete workflow autonomous.

Does the pet choose where to click?

Not in the supplied latest score code. `drive.py` takes coordinates selected by a user or agent. The older brain has pet-position mapping, but the latest driver path can work without using the character as a pointer.

Is it learning progressively?

No progressive-learning implementation is shown. Saved routines, skill extraction, persistent decision memory, and independently executing learned behavior are proposed extensions, not current features.

How are actions checked?

The newer driver waits for a journal entry and reports a screenshot difference. Its own comments warn that no difference does not prove failure. The matching uses command names rather than unique IDs, so result correlation is limited.

What access does the laptop give HQ?

Screen capture and input injection into the interactive user desktop. The listing is not an arbitrary remote-shell endpoint, but mouse and keyboard access can still operate logged-in applications. Tailscale connectivity alone does not establish scoped per-command authorization.

What is verified?

The newer document reports a live browser/Discord demonstration. This revision does not independently verify that run. The user currently reports inaccurate clicking. No precise targeting, measured end-to-end latency, or autonomous task completion is claimed here.

Why use MJPEG?

The implementation serves a simple JPEG stream that `mpv` can display. It keeps only the latest frame rather than a session recording.

What happens when the laptop disconnects?

The brain can report a stale or missing feed. The supplied listings do not establish session-bound commands, expired-action rejection, or safe automatic reconnection behavior.

Code listing status

No source appendix is included in this build summary.